Particle Physics Simulation

Measuring what the memory hierarchy is worth, and proving the physics is right

Olajide Badejo

August 2026

Abstract

This report describes a CUDA particle physics engine that solves two problems: direct all pairs gravitational N-body dynamics, and weakly compressible smoothed particle hydrodynamics with spatial hash neighbour search. It is built around two commitments. Every optimisation is a measured result rather than a claim, and physical correctness is enforced by conservation tests rather than by how the output looks.

The N-body force kernel exists in three variants that isolate one technique each. On an RTX 5070 at 200,000 particles, shared memory tiling takes the step from 88.27 to 45.78 milliseconds, a factor of 1.93, and sustains 17.47 TFLOP/s of single precision arithmetic. Fusing the force evaluation with the integrator, which was expected beforehand to be worth ten to twenty percent, is worth +0.1 percent. That gap is not a disappointing result; it is the most informative one in the report, and Section 5.2 explains why an arithmetic bound problem cannot be helped by removing memory traffic, using hardware counters that put device memory throughput below one percent of peak for all three variants.

All 10 physics gates pass. A two body circular orbit conserves energy to 1.33×10^{-5} over ten thousand velocity Verlet steps against a limit of one part in a thousand; total momentum in a ten thousand particle Plummer sphere holds to 1.23×10^{-8}; the sampled radial density of that sphere matches the analytic Plummer profile at a chi squared p value of 0.4416. The fluid solver runs 100,000 particles at 0.591 milliseconds per step.

Contents

1 Introduction **3**
- 1.1 What this is . . . 3
- 1.2 The two commitments . . . 3
- 1.3 The reference machine . . . 4
- 1.4 How to read this report . . . 4

2 Background **5**
- 2.1 The GPU memory hierarchy . . . 5
- 2.2 Gravitational N-body dynamics . . . 5
  - 2.2.1 Why velocity Verlet . . . 6
  - 2.2.2 The Plummer sphere . . . 6
- 2.3 Smoothed particle hydrodynamics . . . 7
  - 2.3.1 Finding neighbours . . . 7

3 Methodology **8**
- 3.1 The kernel ladder . . . 8
- 3.2 Timing protocol . . . 8
- 3.3 Validation design . . . 9
- 3.4 Reproducibility . . . 10

4 Implementation **11**
- 4.1 Layout and state . . . 11
- 4.2 Separating the physics from the kernels . . . 11
- 4.3 The three force kernels . . . 11
- 4.4 The fluid step . . . 11
- 4.5 Diagnostics without perturbing the run . . . 12
- 4.6 Snapshots and resume . . . 12
- 4.7 The style gate . . . 12

5 Results **13**
- 5.1 The kernel ladder . . . 13
- 5.2 Fusion, expectation against measurement . . . 16
- 5.3 What tiling actually fixes . . . 16
- 5.4 Tile width . . . 19
- 5.5 The fluid solver . . . 19
- 5.6 Physics gates . . . 20

6 Discussion **23**
- 6.1 The most useful result was the one that failed to appear . . . 23
- 6.2 Tiling is not a bandwidth optimisation here . . . 23
- 6.3 Where the specification and the measurement disagreed . . . 23

6.4 Limitations 24

7 Conclusion **25**
7.1 Further work 25

Chapter 1

Introduction

1.1 What this is

This is a particle physics engine written in CUDA. It solves two problems that sit at opposite ends of the memory access spectrum, which is the reason both are here.

The first is direct all pairs gravitational N-body dynamics. Every particle attracts every other, so the force evaluation is $O(N^2)$ arithmetic over $O(N)$ data. Access is perfectly regular and entirely predictable.

The second is smoothed particle hydrodynamics, in which a particle interacts only with the neighbours inside a finite smoothing radius. The work is nearly linear in the particle count, but which particles are neighbours changes every step and has to be discovered before any physics can be evaluated. Access is irregular and data dependent.

A GPU rewards the first pattern and punishes the second, and the techniques that matter are different in each case. Putting both in one engine, behind one configuration format and one integrator, makes the contrast concrete.

1.2 The two commitments

No number without a run. Every performance figure in this report was produced by a program in this repository, recorded to a comma separated file with the commit that produced it, and read back into this document mechanically. There are no hand copied numbers. Where I had an expectation before measuring, the expectation is stated and then compared with what actually happened; in the most interesting case, Section 5.2, the expectation was wrong by more than an order of magnitude and the explanation of why is worth more than the optimisation would have been.

Correctness is enforced, not observed. A particle simulation that is subtly wrong still looks like a particle simulation. Fluid still sloshes and stars still orbit when the force law has a sign error in one term or the neighbour search quietly counts some particles twice. The defence is a set of conservation and distribution tests with numerical thresholds, run on every build, that fail loudly. There are 10 of them and Chapter 5 reports what each one measured.

These two commitments interact. The benchmark builds use `fast math`, which lowers the reciprocal square root to a hardware approximation and is worth having. The physics gates cannot tolerate it: the energy drift gate resolves one part in a thousand and the approximation does not leave that much headroom. Rather than choose, the build compiles the device library twice from identical sources, once with fast math for the benchmarks and once without for the gates. A gate that a compiler flag can relax is not a gate.

1.3 The reference machine

Everything here was measured on one machine: an NVIDIA GeForce RTX 5070 of compute capability 12.0 with 48 streaming multiprocessors on a 192 bit memory bus, driven from Ubuntu under the Windows Subsystem for Linux with CUDA 13.3. Its theoretical peak memory bandwidth is 672.0 GB/s. A streaming copy kernel achieves 549.9 GB/s, or 81.8 percent of that peak, and it is the measured figure rather than the theoretical one that every bandwidth statement in this report is compared against. Quoting utilisation against a peak no kernel can reach flatters every kernel by the same factor while saying nothing about any of them.

1.4 How to read this report

Chapter 2 covers the GPU memory hierarchy and the physics of both solvers, including why velocity Verlet rather than anything else. Chapter 3 describes the kernel ladder, the timing protocol and the design of the validation suite. Chapter 4 describes what was built. Chapter 5 is the measurements. Chapter 6 interprets them, including the results that contradicted my expectations. A companion document, `debug_report.pdf`, records the defects found along the way and how each was diagnosed.

Chapter 2

Background

2.1 The GPU memory hierarchy

The arithmetic units of a modern GPU are far faster than the path that feeds them. The reference card sustains tens of teraflops of single precision arithmetic while its device memory delivers 549.9 GB/s in practice. At four bytes per float that is roughly one hundred and thirty billion floats per second against tens of trillions of operations per second, so a kernel that performs one operation per float loaded runs at well under one percent of the arithmetic the hardware could do. Every technique in this project is a way of raising the number of operations performed per byte brought in from device memory.

Three levels of the hierarchy matter here.

Coalescing. A warp of thirty two threads issues its loads together. When those threads read consecutive addresses, the hardware merges them into a small number of wide transactions. When they read scattered addresses it issues many narrow ones and discards most of what it fetched. This is why the particle state is stored as a structure of arrays rather than an array of structures: if one particle's seven floats sat together, a warp reading the x component of thirty two consecutive particles would touch thirty two separate cache lines and use a fraction of each. With separate arrays the same read is thirty two consecutive floats [7].

Shared memory. Each multiprocessor has a small block of memory, 48 KiB per block on this card, that all threads of a block can read at a latency and bandwidth far better than device memory. A block can cooperatively stage a tile of data there once and then have all its threads read it many times. This is the mechanism behind the tiled force kernel.

Registers and the load store path. Values held in registers cost nothing to read. Between registers and device memory sit the first level cache and the load store pipeline, and as Chapter 5 shows, on this problem that pipeline rather than device memory is what actually limits the naive kernel.

2.2 Gravitational N-body dynamics

The acceleration of body i under the gravity of all the others is

$$\mathbf{a}_i = G \sum_{j \neq i} m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{\left(|\mathbf{r}_j - \mathbf{r}_i|^2 + \varepsilon^2 \right)^{3/2}}. \tag{2.1}$$

The softening length ε in the denominator is not a numerical convenience to be apologised for. Without it the acceleration diverges as two bodies approach, a single time step throws the

pair apart at enormous speed, and the run is destroyed. For a collisionless system, where the smooth mean field rather than individual close encounters carries the dynamics, softening is the physically appropriate model [2]. The potential must be softened consistently,

$$U = -G \sum_{i<j} \frac{m_i m_j}{\sqrt{|\mathbf{r}_j - \mathbf{r}_i|^2 + \varepsilon^2}}, \tag{2.2}$$

because pairing a softened force with an unsoftened potential breaks energy conservation for a reason that has nothing to do with the integrator. Both come from the same expression in the code, and a test differentiates Equation 2.2 numerically and compares it against Equation 2.1.

2.2.1 Why velocity Verlet

The integrator advances

$$\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_n \Delta t + \tfrac{1}{2}\mathbf{a}_n \Delta t^2, \tag{2.3}$$

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \tfrac{1}{2}\left(\mathbf{a}_n + \mathbf{a}_{n+1}\right)\Delta t. \tag{2.4}$$

Three alternatives were considered and rejected.

Explicit Euler is first order and not symplectic. Its energy error grows without bound, and on a circular orbit it spirals outward visibly within a few hundred steps.

Fourth order Runge Kutta is accurate per step but requires four force evaluations per step. Since the force evaluation is essentially the entire cost of this simulation, that is a factor of four in exchange for accuracy that a symplectic method achieves differently.

Velocity Verlet is second order, needs exactly one force evaluation per step, and is symplectic: it exactly conserves a modified Hamiltonian near the true one, so its energy error oscillates within a bound rather than accumulating [11, 4]. Over ten thousand steps that distinction is the difference between an orbit that is still an orbit and one that is not. Figure 5.6 shows the bounded behaviour directly.

The fused kernel uses the leapfrog form,

$$\mathbf{v}_{n+1/2} = \mathbf{v}_{n-1/2} + \mathbf{a}_n \Delta t, \tag{2.5}$$

$$\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_{n+1/2} \Delta t, \tag{2.6}$$

which produces the same position sequence to rounding, provided the velocity is offset backwards by half a step once before the loop begins. That offset, and the fact that the stored velocity afterwards is not synchronised with the positions, is a detail with real consequences for snapshots and for the energy diagnostic; both are handled explicitly in Chapter 4.

2.2.2 The Plummer sphere

Benchmarks need a physically sensible initial condition. A uniform random cloud is not one: it is far from equilibrium and collapses immediately, so it measures the wrong thing. The Plummer sphere [9] has density

$$\rho(r) = \frac{3M}{4\pi a^3}\left(1 + \frac{r^2}{a^2}\right)^{-5/2}, \qquad \frac{M(r)}{M} = \frac{r^3}{\left(r^2 + a^2\right)^{3/2}}, \tag{2.7}$$

and the cumulative profile inverts in closed form, so radii are sampled directly. Speeds are drawn from the exact distribution function by the rejection method of Aarseth, Hénon and Wielen [3], which puts the sphere in virial equilibrium at $t = 0$. Drawing speeds from a convenient Maxwellian instead would leave it out of equilibrium, and it would visibly breathe on the first crossing time in a way that looks exactly like an integrator bug.

2.3 Smoothed particle hydrodynamics

SPH represents a continuous field by interpolating over particles with a smoothing kernel W of finite support h [5]. Density is

$$\rho_i = \sum_j m_j W(|\mathbf{r}_i - \mathbf{r}_j|, h), \tag{2.8}$$

and pressure follows from a stiff equation of state $p = k(\rho - \rho_0)$, which keeps the fluid weakly compressible without solving a pressure Poisson equation every step.

Three different kernels are used, each for the one job it does well [6]. The *poly6* kernel is a polynomial in r^2, so density needs no square root. The *spiky* kernel supplies the pressure gradient, because its gradient does not vanish as the separation goes to zero; the poly6 gradient does, which means no restoring force exactly where the fluid needs one most, and particles pile up on top of each other. The *viscosity* kernel has a positive Laplacian throughout its support, so the viscous term always damps and never injects energy. Each integrates to one over its support, which is the property the normalisation gate checks by quadrature.

2.3.1 Finding neighbours

The neighbour set changes every step and must be rebuilt every step. A dense uniform grid needs storage proportional to the bounding volume rather than to the particle count, and a dam break spends most of its life with the fluid occupying a small and moving part of the domain, so most of that grid is empty. A k-d tree is a poor fit for a full rebuild on a GPU every step.

Spatial hashing [10] maps the integer cell coordinate to a table sized by particle count:

$$\text{hash}(i, j, k) = (i\, p_1 \oplus j\, p_2 \oplus k\, p_3) \bmod n, \tag{2.9}$$

with $p_1 = 73856093$, $p_2 = 19349663$, $p_3 = 83492791$ and n prime.

Hashing is many to one, and that has a consequence which is easy to miss. Two different cells can land in the same bucket. The neighbour search visits the twenty seven cells surrounding a particle, and two of those cells can hash to the same bucket, in which case a naive scan counts the particles in it twice. Distance filtering does not save you: those particles are genuinely within the smoothing radius, they are just being counted more than once. The engine therefore stores an exact 64 bit cell key per particle and compares it during the scan. Section 5.5 reports the measured agreement with a brute force sum, which is what proves the defence works.

Chapter 3

Methodology

3.1 The kernel ladder

The point of the ladder is attribution. Three force kernels differ by exactly one technique each, so the difference between two rungs can be assigned to that technique and nothing else. All three compute the same quantity from the same structure of arrays layout, and a test asserts they agree with a double precision host reference to better than one part in ten thousand, so a performance difference cannot be a difference in what is being computed.

Rung one, naive. Every thread walks the whole particle array in device memory. This is not a straw man built to lose. All threads in a warp read the same index on the same cycle, so the load is a broadcast and the line stays resident for the whole warp. What it does not do is share that line between the warps of a block, so the same data crosses the cache boundary once per resident block rather than once per tile.

Rung two, tiled. The block cooperatively stages a tile of source particles in shared memory and every thread of the block consumes it, following Nyland, Harris and Prins [8]. The tile width is thirty two by default. The cost is two barriers per tile, so narrow tiles pay the barrier more often per unit of arithmetic; Section 5.4 measures that trade rather than arguing about it.

Rung three, fused. The tiled force evaluation and the integrator update happen in one launch. This removes a round trip of the acceleration array, $12N$ bytes written and $12N$ bytes read, plus one kernel launch per step.

Fusion has a correctness constraint that is worth stating because it is easy to get wrong. There is no grid wide barrier inside a launch. If the kernel updated positions in place, a block that had finished its force sum could overwrite a position that another block was still reading, and the result would depend on scheduling. Positions are therefore double buffered: read from one array, written to the other, swapped afterwards. This costs $12N$ bytes of memory and no bandwidth at all, because the write has to happen either way.

3.2 Timing protocol

Three things ruin a GPU timing measurement, and the protocol addresses each.

The first launch of a kernel pays for module loading and for the card ramping up its clocks, so every measurement is preceded by warm up steps that are not timed.

The host can queue work far faster than the device retires it, so timing on the host measures the host. All timings use CUDA events recorded on the same stream as the work.

Run length varies enormously across the sweep, from a fraction of a millisecond to nearly a hundred. Each point is therefore calibrated first with a single timed step, and the run length is chosen to make each trial last about 120 ms.

Each point runs three such trials and the fastest is kept. The minimum rather than the mean, because everything that perturbs a measurement, whether thermal throttling or another process on the display, makes it slower and never faster. The minimum is the estimate least contaminated by whatever else the machine was doing.

One caveat is worth recording. Absolute timings depend on the thermal state of the card. A tile width sweep run immediately after the full benchmark sweep produced timings about fifteen percent higher than the same sweep run from cold, while the relative ordering and the percentage differences between tile widths were reproduced to within a tenth of a percentage point. The comparisons in this report are therefore all made within a single session, and the conclusions rest on relative differences rather than on absolute milliseconds.

3.3 Validation design

Each gate is chosen to fail for a specific class of defect rather than to produce a reassuring number.

Two body energy drift. A circular orbit integrated for ten thousand steps must conserve total energy to better than one part in a thousand. This is the sharpest single statement that can be made about an integrator, and it also catches an inconsistent softened potential and a mis-derived initial orbital speed. The initial speed is computed from the softened force law, so the orbit is genuinely circular rather than a slowly precessing ellipse.

Momentum conservation. The internal forces of an isolated system cancel exactly in the mathematics. On the device they cancel only to rounding, because each thread accumulates its own sum in single precision in its own order. The gate bounds the residual against what single precision can explain, $\epsilon_{\mathrm{f32}} \sqrt{N} n_{\mathrm{steps}}$, rather than against an arbitrary constant.

Force antisymmetry. Newton's third law checked pair by pair against an independent double precision host implementation, written as literally as the force law can be written, with no tiling and no reciprocal square root. Its only job is to be obviously correct.

Plummer radial density. A chi squared test of the sampled radial distribution against the analytic cumulative mass profile. Shells are chosen to hold equal expected counts, so no bin is too sparse for the statistic to be valid. A sampler with a subtly wrong exponent produces a blob that looks perfectly reasonable and fails this immediately. The test is itself checked by running it against a deliberately wrong scale radius, which it must reject.

SPH kernel normalisation. Each smoothing kernel is integrated over its support by composite Simpson quadrature and must come to one within 10^{-3}. A kernel whose normalisation is off by a few percent produces a density field wrong by the same few percent everywhere, and the fluid still looks like a fluid.

Neighbour search against brute force. The density field from the hashed neighbour search is compared against an $O(N^2)$ host sum over the same positions. This is what would catch a dropped or double counted neighbour, and it is the direct test of the cell key defence described in Chapter 2.

3.4 Reproducibility

Every configuration carries an explicit seed and the initial conditions are generated on the host from a written out uniform mapping rather than from a standard library distribution, whose sequence is not specified across implementations. Each benchmark row records the seed, the commit, the CUDA runtime and driver versions, the fast math setting and a timestamp. Snapshots are written to a temporary file and renamed into place, so an interrupted run never leaves a half written snapshot for a later resume to pick up, and a test asserts that stopping and resuming reproduces a straight through run bit for bit.

Chapter 4

Implementation

4.1 Layout and state

Particle state lives on the device for the whole run as thirteen float arrays: three positions, three staged positions for the fused path, three velocities, three accelerations and the mass. At two hundred thousand particles that is about ten megabytes, so memory is never a constraint at these sizes and the layout decision is entirely about access pattern.

The host holds no particle data during a run. Configuration and diagnostic accumulators are the only host allocations, and the snapshot staging buffer is touched only when a snapshot is actually written.

4.2 Separating the physics from the kernels

The force law, the smoothing kernels, the equation of state and the spatial hash all live in headers marked so they compile both as device code and as ordinary host C++. This buys two things. The kernels and the host side tests cannot drift apart, because they are the same code. And the formula level tests, which are the majority of them, run in continuous integration on a machine with no GPU.

4.3 The three force kernels

The tiled kernel has one subtlety worth recording. Threads whose index is past the end of the array must not return early, because they still have to reach both barriers on every iteration of the tile loop; a barrier that only some threads of a block arrive at is undefined behaviour. They run the whole loop with clamped reads and discard the result at the end. Padded tile slots carry zero mass, so they contribute nothing and the inner loop needs no bounds test at all. Self interaction is likewise harmless and is deliberately not special-cased: the numerator vanishes while the softened denominator stays finite, so the diagonal term contributes exactly zero and the inner loop stays branch free.

4.4 The fluid step

One fluid step is six device passes: hash, sort, cell bounds, gather, density, and a combined force and integration pass.

The sort is a CUB radix sort of bucket and index pairs [1]. Only the bits that can differ are sorted: the table size bounds the bucket index, so the sort stops at the significant bit count rather than walking all thirty two bits.

After sorting, particle data is gathered into cell sorted order, and the permutation is kept. The particle arrays end a step in the order the next step wants, so the sort has an effect that outlives the step that paid for it. Because that permutation must be a bijection, one gate checks that the multiset of particle masses is unchanged across a step and another checks that total mass is conserved over two thousand steps; a gather that lost or duplicated an index would show up in both.

The force and integration pass is fused for the same reason the gravitational one is, and it is safe for the same reason: the sorted arrays it reads are a separate copy from the primary arrays it writes.

4.5 Diagnostics without perturbing the run

The particle state is single precision, but the reduction that measures conservation is not. A float32 tree reduction over ten thousand terms loses three to four decimal digits, and the energy gate resolves one part in a thousand over ten thousand steps; reducing in single precision would have measured the reduction rather than the integrator. Accumulators are double, and the block partials are summed in a fixed order on the host rather than with atomics, so two runs of the same configuration agree bit for bit.

The fused variant carries the leapfrog half step velocity, which is not the velocity belonging to the current positions. Recovering the synchronised velocity by shifting the array forward and then back would not return the original bits, and that would break the bit identity guarantee for any run that happened to take a diagnostic. The correction is applied inside the reduction kernel instead, and a test asserts that taking a diagnostic on every step leaves the trajectory unchanged.

4.6 Snapshots and resume

The snapshot header is sixteen bytes: magic number, particle count, step index, and a flags word. The flags word carries the one thing a reader cannot infer from the numbers, namely whether the stored velocities are synchronised with the positions or offset by half a step. Resuming a fused run under the wrong convention would shift the whole trajectory by half a step, which looks exactly like a subtly wrong force.

Accelerations are not stored. They are a deterministic function of the positions, so recomputing them on load is cheaper than storing them and guarantees the resumed run enters the loop in exactly the state the original run held.

4.7 The style gate

The repository forbids en dashes and em dashes. Enforcing that on source files is easy; enforcing it on the reports is not, because TeX silently turns two hyphens into an en dash. The gate therefore also parses the LaTeX sources for that ligature, excluding verbatim regions where a command line flag legitimately contains two hyphens, and runs over the compiled PDFs through a text extractor. It caught a real instance during this build: the page ranges in the bibliography, written in the conventional BibTeX style, would have rendered as en dashes in the reference list of the finished document.

Chapter 5

Results

Every number in this chapter was read mechanically out of a file written by a run at commit `8e4c710216be`, on the machine described in Chapter 2, with fast math on for the benchmark library and off for everything in Section 5.6.

5.1 The kernel ladder

	milliseconds per step			speedup		tiled
particles	naive	tiled	fused	tiled	fused	TFLOP/s
10,000	0.482	0.225	0.221	2.14	2.19	8.88
20,000	1.098	0.624	0.626	1.76	1.76	12.83
50,000	6.860	3.586	3.592	1.91	1.91	13.94
100,000	23.773	12.296	12.498	1.93	1.90	16.27
150,000	51.221	26.380	26.378	1.94	1.94	17.06
200,000	88.265	45.781	45.724	1.93	1.93	17.47

Table 5.1: The N-body ladder. Milliseconds per step for each variant, the speedup of each against the naive baseline, and the sustained arithmetic throughput of the tiled kernel.

	naive to tiled		tiled to fused	
particles	time saved	speedup	time saved	speedup
10,000	+53.3%	2.141	+2.1%	1.021
20,000	+43.2%	1.761	-0.3%	0.997
50,000	+47.7%	1.913	-0.2%	0.998
100,000	+48.3%	1.933	-1.6%	0.984
150,000	+48.5%	1.942	+0.0%	1.000
200,000	+48.1%	1.928	+0.1%	1.001

Table 5.2: What each rung of the ladder is worth on its own. The first pair of columns is the effect of shared memory tiling; the second is the effect of fusing the integrator into the force kernel.

Tiling is worth a factor of 1.93 at 200,000 particles, taking the step from 88.27 to 45.78 milliseconds. The advantage grows with particle count, which is what the mechanism predicts: the benefit comes from reuse within a block, and at small counts there are not enough blocks resident to matter and the launch overhead is a larger share of the step.

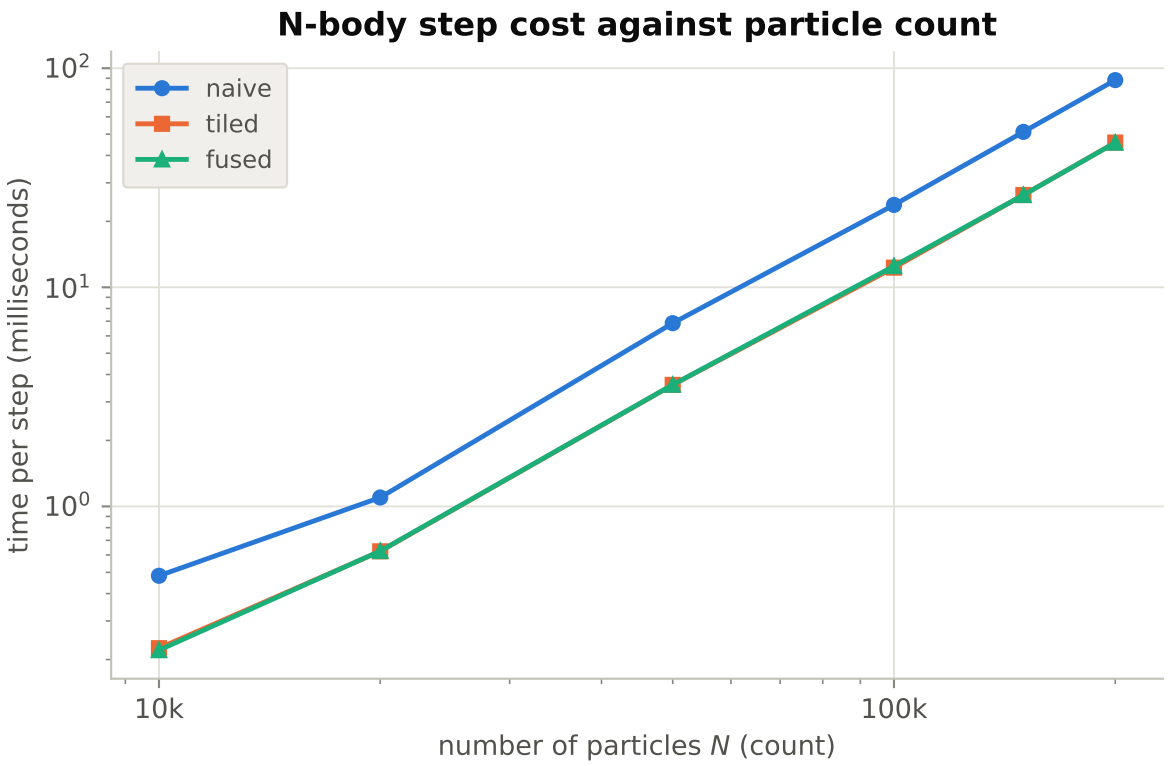


Figure 5.1: Step cost against particle count for the three variants.

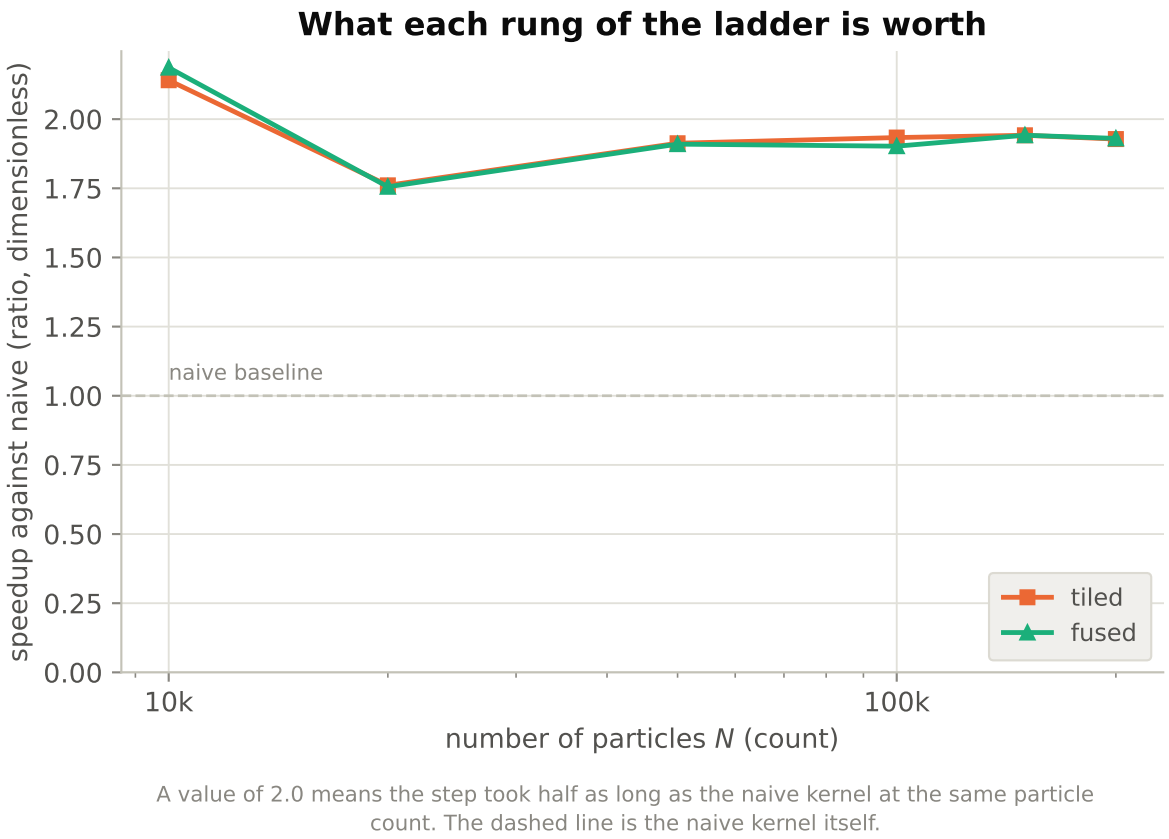


Figure 5.2: The same data as a speedup against the naive baseline. The tiled and fused lines lie almost on top of each other, which is itself the result of Section 5.2.

The tiled kernel sustains 17.47 TFLOP/s at the largest size, and the peak observed anywhere in the sweep is 17.50 TFLOP/s.

5.2 Fusion, expectation against measurement

Before measuring, I expected fusion to be worth ten to twenty percent. It is worth +0.1 percent at 200,000 particles and +2.1 percent at 10,000. The expectation was wrong by more than an order of magnitude, and the reason is worth more than the optimisation would have been.

The arithmetic settles it before any profiler is involved. Fusion removes a round trip of the acceleration array: $12N$ bytes written and $12N$ bytes read, which at 200,000 particles is 0.0087 milliseconds at the measured streaming bandwidth of 549.9 GB/s. Against a step of 45.78 milliseconds that is 0.019 percent. That is not what fusion did deliver; it is the absolute ceiling on what fusion *could* deliver. An optimisation that removes memory traffic cannot help a problem that is not waiting on memory.

The hardware counters confirm it directly.

	throughput, percent of peak			per launch	
variant	L1 and load store	L2	device memory	global load sectors	warp issue
naive	80.19	1.77	0.55	1,250.0M	50.12
tiled	39.93	1.76	0.45	19.6M	74.53
fused	40.15	1.80	0.57	19.6M	74.94

Table 5.3: Nsight Compute counters for one force kernel launch at one hundred thousand particles. Device memory throughput is below one percent of peak for every variant.

Device memory throughput is 0.55 percent of peak for the naive kernel and 0.45 percent for the tiled one. The all pairs sum is $O(N^2)$ arithmetic over $O(N)$ data, so the working set is small, it stays resident in cache, and device memory is almost idle throughout. There was never any device memory pressure for fusion to relieve.

5.3 What tiling actually fixes

Table 5.3 also corrects a second assumption. Tiling is usually described as a bandwidth optimisation, and on this problem it is not one: device memory throughput is negligible both before and after. What tiling reduces is pressure on the first level cache and the load store pipeline.

Global load sectors fall by a factor of 63.8 and load store instructions by a factor of 3.8. First level cache throughput falls from 80.2 percent of peak, which is close to saturation and is the real bottleneck of the naive kernel, to 39.9 percent. The consequence shows up in warp issue efficiency, which rises from 50.1 percent to 74.5 percent: the naive kernel spends about half its time with nothing to issue because its warps are waiting on the load store pipe.

The arithmetic instruction counts are the control in this experiment. They are equal across the three variants to within a fraction of a percent, confirming that the variants differ only in how they move data. Those counters also let the conventional operation count be checked rather than assumed: the twenty floating point operations per interaction used throughout this report compares with 18.0 actually retired, counting a fused multiply add as two.

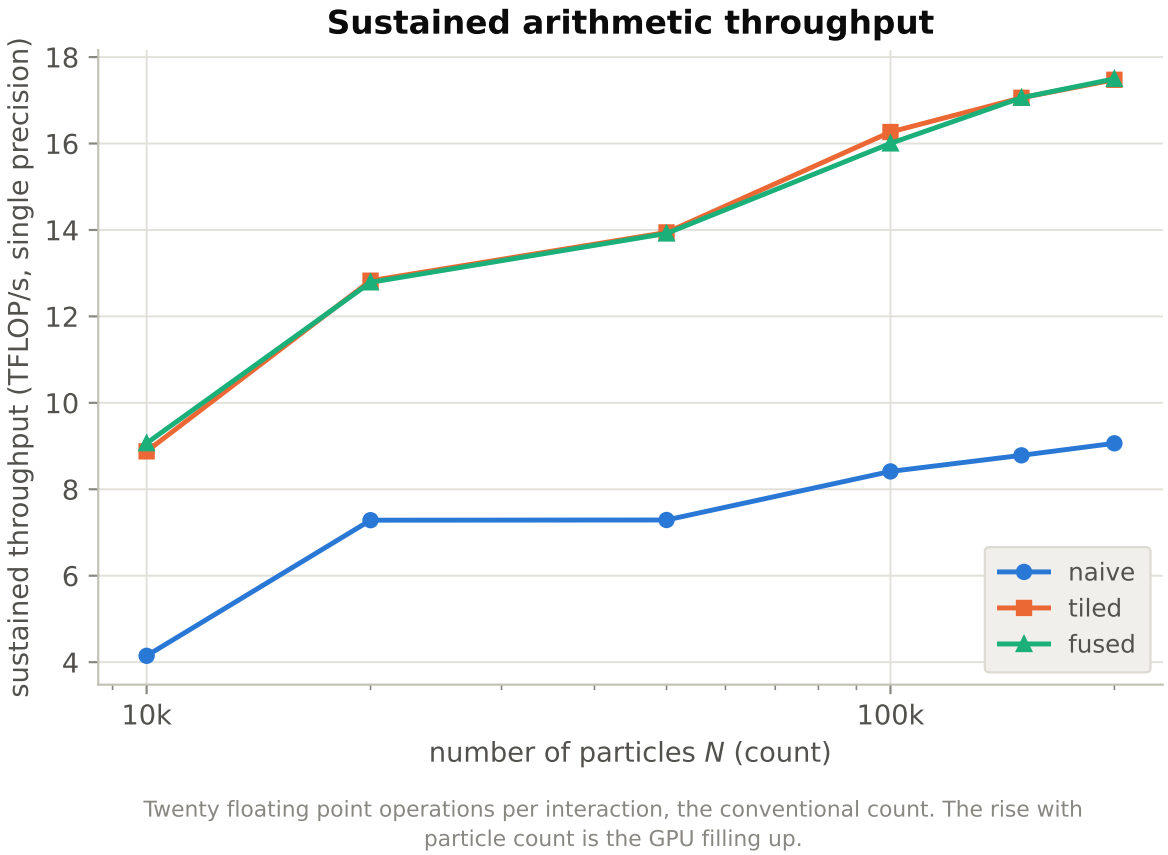


Figure 5.3: Sustained single precision throughput. The rise with particle count is the GPU filling up.

	tiled		fused	
tile width	ms per step	TFLOP/s	ms per step	TFLOP/s
32	12.658	15.80	13.096	15.27
64	12.996	15.39	12.023	16.63
128	11.911	16.79	11.812	16.93
256	11.842	16.89	11.758	17.01

Table 5.4: Tile width sweep at one hundred thousand particles.

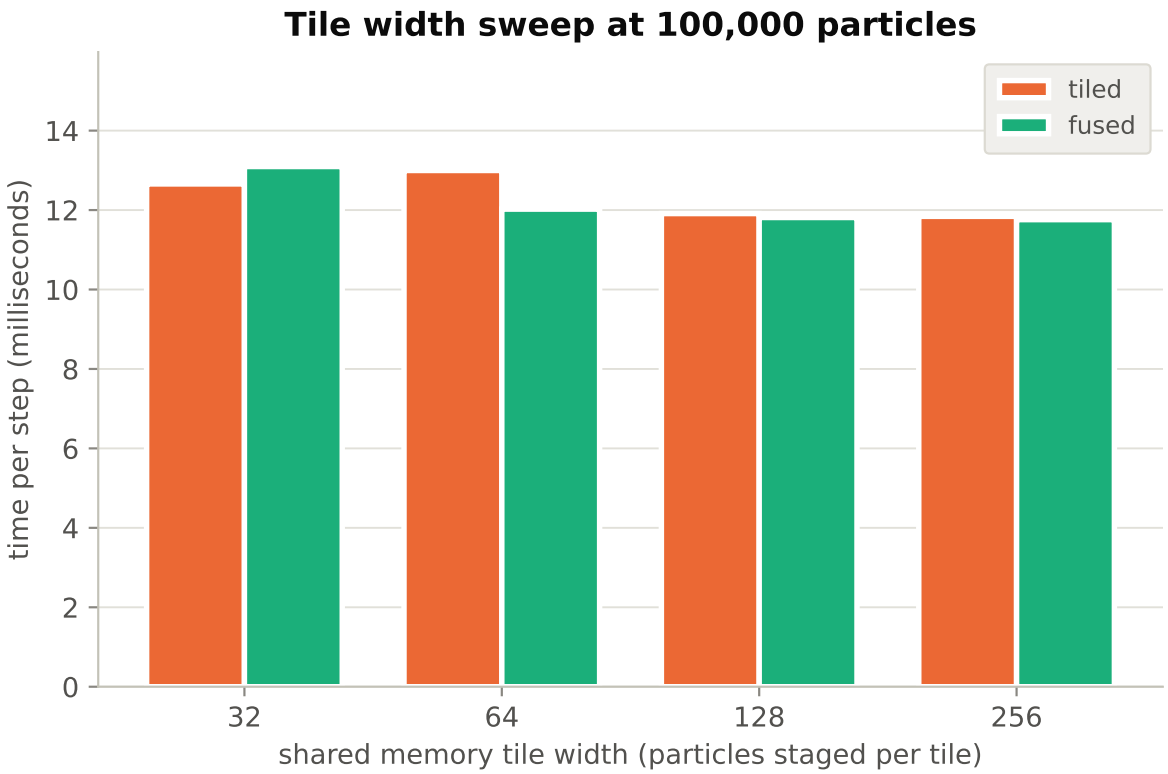


Figure 5.4: Step cost against shared memory tile width.

5.4 Tile width

The specified tile width of thirty two is not the fastest on this card. A width of 256 runs at 11.842 milliseconds against 12.658 for a width of thirty two, a difference of 6.4 percent. The mechanism is the barrier count: two barriers are paid per tile, so a narrow tile pays them more often per unit of arithmetic. The default remains thirty two because that is what the specification names, and changing it would have made the ladder measurements incomparable with the design they were meant to test; the measurement is recorded here so the choice is an informed one.

5.5 The fluid solver

particles	milliseconds per step	steps per second	million particles per second
10,000	0.2520	3,969	39.7
25,000	0.2673	3,740	93.5
50,000	0.3375	2,963	148.1
75,000	0.5396	1,853	139.0
100,000	0.5906	1,693	169.3

Table 5.5: Fluid step cost against particle count.

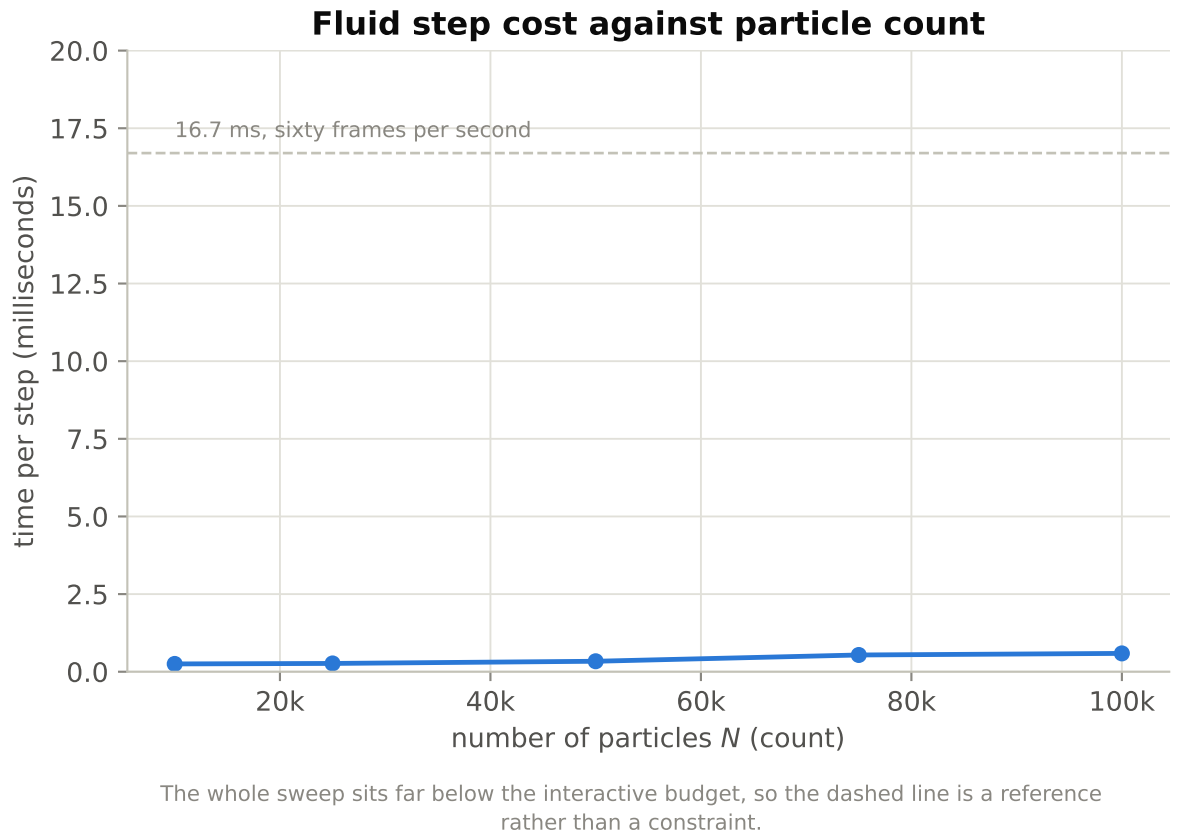


Figure 5.5: Fluid step cost against particle count.

At the fifty thousand particles the specification targets, a step costs 0.337 milliseconds, or 2,963 steps per second. At 100,000 particles it costs 0.591 milliseconds, 1,693 steps per second.

The target of interactive rates at fifty thousand particles is met with a margin of roughly fifty times against a sixty frame per second budget.

Scaling is markedly sublinear at the small end: ten times the particles costs about two and a half times the time. The fixed costs of the step, clearing a hash table sized by particle count and the six kernel launches, dominate when there is little work to spread them over.

5.6 Physics gates

gate	measured	limit	verdict
Two body orbit, relative energy drift over 10,000 steps	1.33×10^{-5}	< 0.0010	**pass**
Two body orbit, departure from circular	9.35×10^{-6}	< 0.0010	**pass**
Plummer 10k, relative momentum drift over 1,000 steps	1.23×10^{-8}	< 0.0119	**pass**
Force against a double precision CPU reference, 100 particles	5.77×10^{-7}	$< 1.00 \times 10^{-5}$	**pass**
Pairwise force antisymmetry residual	1.53×10^{-7}	$< 1.00 \times 10^{-6}$	**pass**
Plummer radial density, chi squared p value	0.4416	> 0.0500	**pass**
SPH poly6 kernel normalisation error	6.48×10^{-8}	< 0.0010	**pass**
SPH spiky kernel normalisation error	4.39×10^{-8}	< 0.0010	**pass**
SPH interior density, relative error	0.0420	< 0.0500	**pass**
SPH neighbour search against a brute force sum	3.20×10^{-7}	$< 1.00 \times 10^{-4}$	**pass**

Table 5.6: Every physics gate, its measured value and its limit. All values come from the artifacts written by the validation suite, which links the library compiled with fast math off.

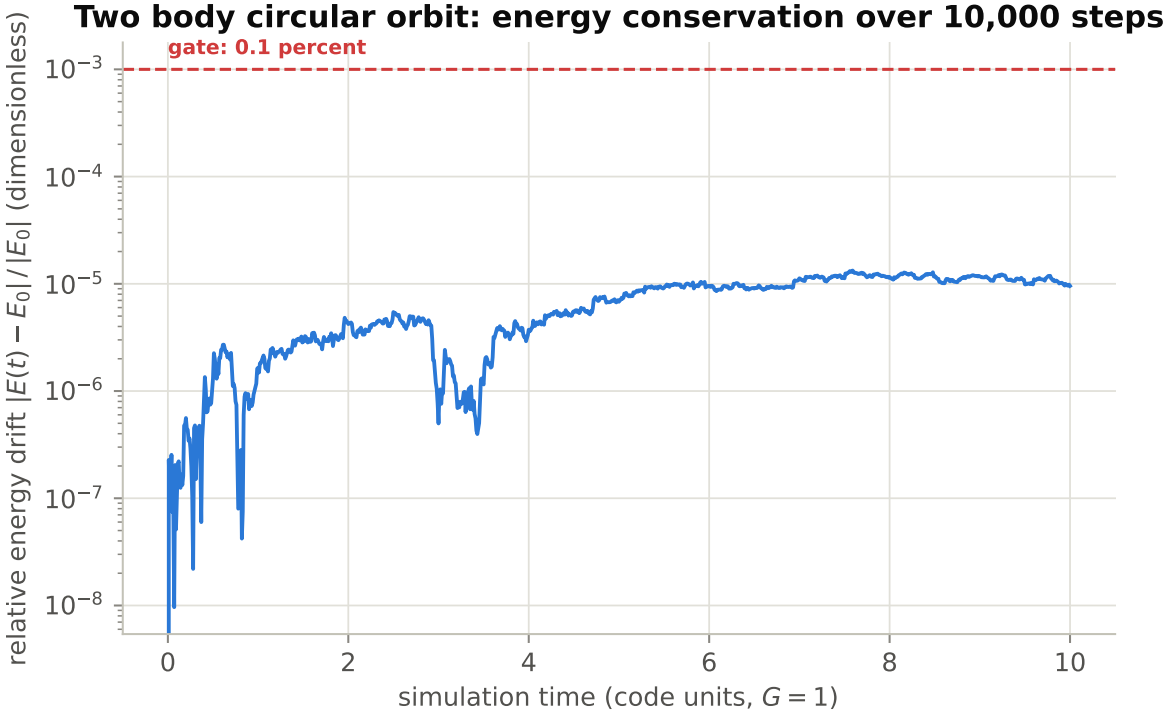
All 10 gates pass, most with several orders of magnitude of margin.

The energy drift measured over ten thousand steps is 1.33×10^{-5}, seventy five times inside the limit of one part in a thousand. Figure 5.6 shows the shape that matters as much as the magnitude: the error rises and then settles onto a bounded plateau rather than growing without limit. That is the property a symplectic integrator guarantees. The test also compares the worst error in the first half of the run against the second, so a scheme that had begun to accumulate would fail even if its total error were still small.

Momentum holds to 1.23×10^{-8} relative to the total absolute momentum in the system, six orders of magnitude inside the bound that single precision arithmetic can explain.

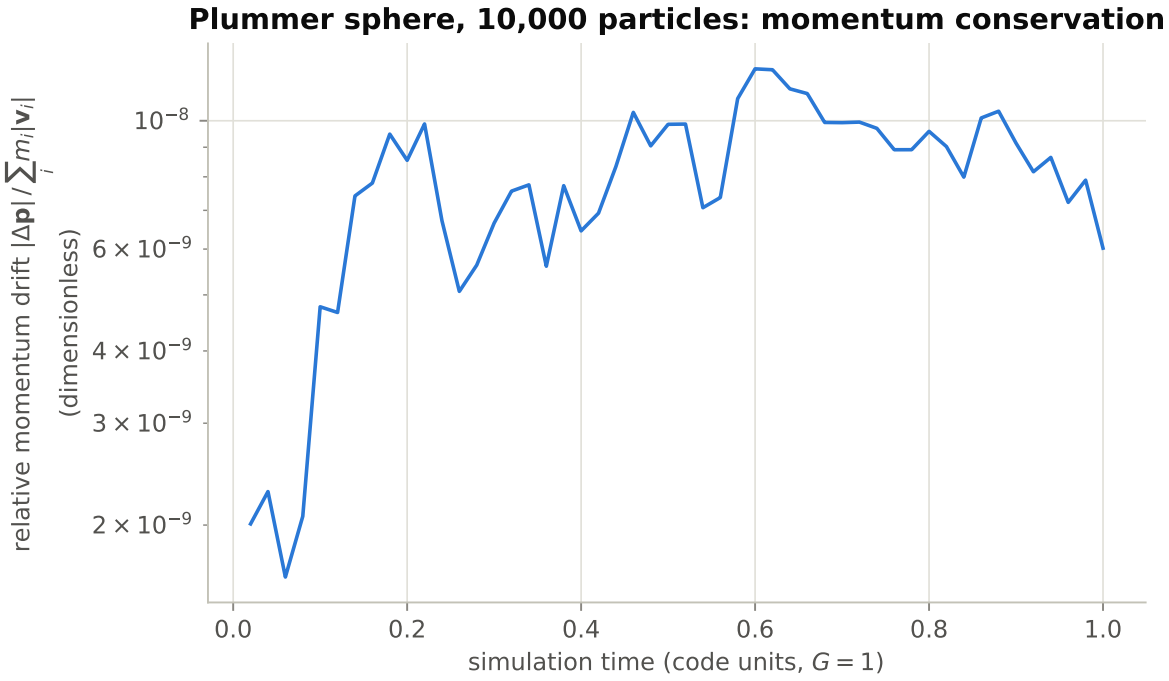
The radial density fits the analytic profile at a chi squared p value of 0.4416, comfortably above the threshold of 0.05. The same test applied to a model whose scale radius is ten percent wrong returns a p value below 10^{-6}, which is what establishes that the test can reject at all.

For the fluid solver, the poly6 kernel integrates to one with an error of 6.48×10^{-8}, and the density field from the hashed neighbour search agrees with a brute force $O(N^2)$ sum over the same positions to three parts in ten million. The interior of a block filled at rest density recovers that density to 4.20 percent. The residual is the expected consequence of a finite smoothing radius on a jittered lattice rather than a defect: the density estimate is a weighted average over a finite neighbourhood, and near the free surface it legitimately falls short because there is no fluid on the outside.



Time is in the code units of the run, where the gravitational constant, the total mass and the initial separation are all one, so one orbit is about 4.44 units. The error rises and then settles onto a bounded plateau instead of growing without limit, which is the property a symplectic integrator guarantees and a non symplectic one does not.

Figure 5.6: Energy conservation of the two body circular orbit.



Drift is measured against the total absolute momentum in the system, because the vector total is zero by construction and dividing by it would be dividing by noise. The rise is single precision rounding accumulating in the force sum.

Figure 5.7: Momentum conservation of a ten thousand particle Plummer sphere.

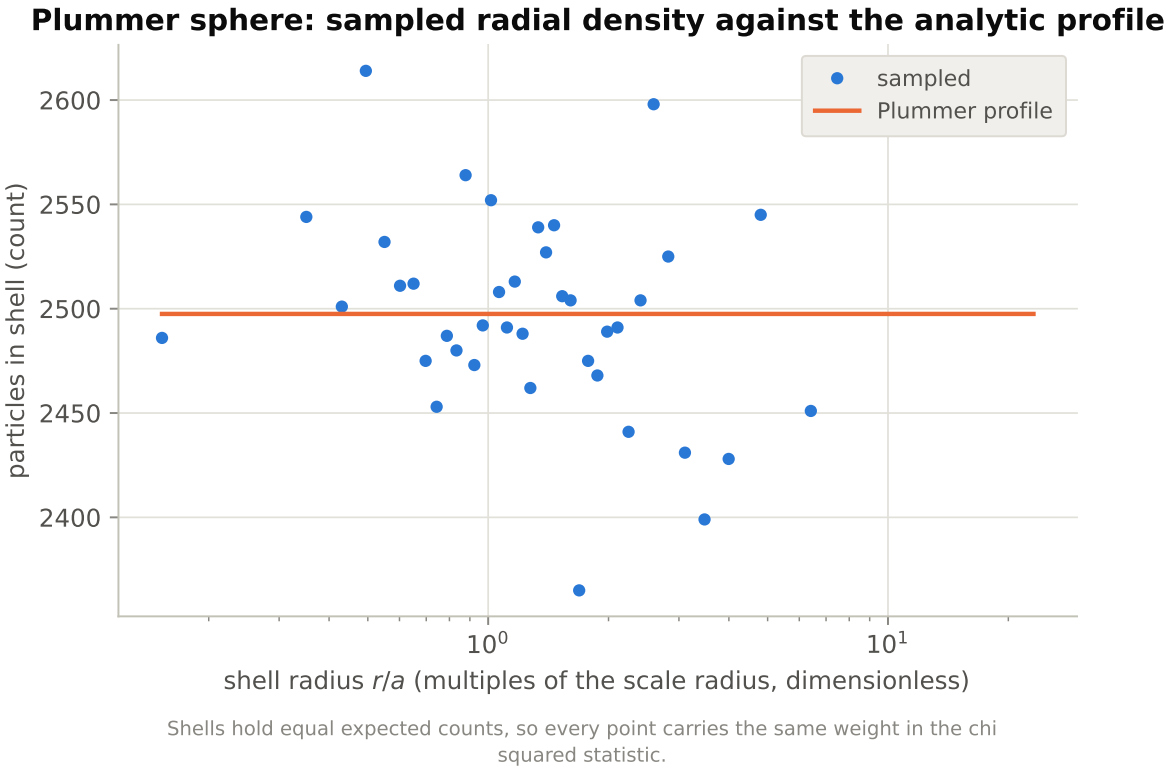


Figure 5.8: Sampled radial density of the Plummer sphere against the analytic profile.

Chapter 6

Discussion

6.1 The most useful result was the one that failed to appear

The fusion measurement is the part of this project I would keep if I could keep only one thing. The expectation of ten to twenty percent was not arbitrary; it is roughly right for a memory bound kernel, and removing a $24N$ byte round trip per step sounds substantial when stated in bytes. It is worth +0.1 percent.

What the expectation missed is arithmetic intensity. The all pairs sum performs $O(N^2)$ arithmetic on $O(N)$ data, so the ratio of operations to bytes grows without limit as the problem grows. At 200,000 particles the force evaluation performs about 4×10^{11} floating point operations while touching a working set of a few megabytes. Removing memory traffic from a kernel in that regime is removing something that was not costing anything: the counters put device memory throughput at 0.45 percent of peak.

The general lesson is that an optimisation should be sized against the resource it relieves before it is implemented, not after. Two minutes of arithmetic with the measured bandwidth figure would have predicted 0.019 percent as an upper bound and saved the effort of being surprised. That said, the fused kernel is not wasted: it is the correct choice at small particle counts, where it wins +2.1 percent because launch overhead is then a real fraction of the step, and it halves the memory footprint of a step.

6.2 Tiling is not a bandwidth optimisation here

The second correction is subtler. Shared memory tiling is conventionally introduced as a way of reducing device memory traffic, and the introduction to this report frames it that way. On this problem the framing is wrong, and the counters say so: device memory throughput is under one percent of peak both before and after the change. Tiling still delivers a factor of 1.93, but the resource it relieves is the first level cache and the load store pipeline, not the memory bus. Global load sectors fall by a factor of 63.8, and the naive kernel's first level cache throughput of 80.2 percent of peak identifies it as the true bottleneck.

This matters practically. Had I tried to improve the naive kernel by attacking device memory traffic, for instance by compressing the state or by adding prefetching, nothing would have happened. The profile pointed at the right resource; reasoning from the textbook description would not have.

6.3 Where the specification and the measurement disagreed

Three points in the original design were changed or contradicted by measurement, and each is recorded rather than quietly absorbed.

The host compiler floor of GCC 16.1 is not achievable with CUDA 13.3, which refuses any GCC newer than 15. GCC 15.2 supplies the C++23 host standard the floor was really for.

The tile width of thirty two costs 6.4 percent against a width of 256 on this card. The default was kept so the ladder would measure the design as specified, and the sweep is reported so the choice is informed.

The estimated runtimes in the specification are far too long. The full validation suite, which was estimated at ten to twenty minutes on the grounds that the ten thousand step orbit would dominate, completes in about five seconds. The estimate assumed the orbit run would be expensive; with two particles it is trivially cheap, and the cost is dominated instead by the ten thousand particle Plummer runs. The full benchmark sweep, estimated at fifteen to thirty minutes, completes in under two minutes.

6.4 Limitations

The all pairs solver is $O(N^2)$ and always will be. At two hundred thousand particles a step costs 45.78 milliseconds; at two million it would cost a hundred times that. The range where this approach is the right one is bounded, and beyond it a tree code is not an optimisation but a different algorithm.

The fluid solver is weakly compressible, so it enforces incompressibility only through a stiff equation of state. That bounds the time step: the sound speed implied by the stiffness constant sets a Courant condition far tighter than the fluid velocity would. The dam break runs at a time step of 10^{-4} seconds for that reason, so the interactive step rates reported in Section 5.5 are steps per second and not seconds of simulated time per second.

Single precision is a deliberate choice and it has a cost. The momentum gate measures it directly: the residual is exactly the accumulation of rounding in the force sum. For the problems here that cost is far inside the tolerances, but a long term integration of a planetary system would need double precision positions, and the ladder would look different because double precision arithmetic is much slower relative to memory on consumer hardware.

The benchmark measures the fluid from a configuration that begins as a settled block and evolves for a few hundred steps during measurement. A fully developed splash has a different spatial distribution and therefore a different neighbour list length, so the fluid timings should be read as representative of the early collapse rather than of every phase of the flow.

Chapter 7

Conclusion

The engine does what it was built to do. Direct N-body gravity runs at 17.47 TFLOP/s of sustained single precision arithmetic at 200,000 particles, and the fluid solver runs 100,000 particles at 0.591 milliseconds per step, roughly fifty times inside an interactive budget at the fifty thousand particles it was aimed at. All 10 physics gates pass, most with orders of magnitude of margin, against a library compiled with fast math disabled so that no compiler flag can relax them.

The memory hierarchy ladder produced a clear answer and a clear correction. Shared memory tiling is worth a factor of 1.93, and the hardware counters attribute that gain to a 63.8 times reduction in global load sectors relieving a saturated load store pipeline, not to device memory bandwidth, which never rose above one percent of peak in any variant. Fusing the integrator into the force kernel was expected to be worth ten to twenty percent and is worth +0.1 percent, for the good reason that an $O(N^2)$ arithmetic problem on $O(N)$ data is not waiting on memory at all. Both results are more useful than the numbers I expected to report, because both changed how I would size the next optimisation before writing it.

The parts of the design that survived contact with measurement did so because they were chosen against a specific failure. Velocity Verlet was chosen because its energy error is bounded rather than merely small, and Figure 5.6 shows exactly that shape. The exact cell key comparison in the neighbour search was added because hashing is many to one and a bucket scan would otherwise count some neighbours twice, and the brute force comparison confirms it works to three parts in ten million. Double precision diagnostic accumulators were chosen because a single precision reduction would have measured itself rather than the integrator.

7.1 Further work

A Barnes Hut tree code is the obvious next step and the one that changes the reachable problem size rather than the constant factor. Its irregular traversal is a genuinely different GPU problem, and the tiled kernel here would remain the right choice for the leaf level interactions.

The fluid solver would benefit more from a position based or projection based incompressibility treatment than from any further kernel tuning, because the stiff equation of state, not the neighbour search, is what bounds the time step.

The tile width result suggests a small piece of unfinished business: with the default at thirty two for fidelity to the specification, and 256 measurably faster, the right resolution is to make the width a configuration field so the ladder can be run at the specified width while production runs use the fastest one.

Bibliography

[1] CUDA core compute libraries: CUB device wide primitives, 2026.

[2] Sverre J. Aarseth. *Gravitational N-Body Simulations: Tools and Algorithms*. Cambridge University Press, 2003.

[3] Sverre J. Aarseth, Michel Hénon, and Roland Wielen. A comparison of numerical methods for the study of star cluster dynamics. *Astronomy and Astrophysics*, 37:183 to 187, 1974.

[4] Ernst Hairer, Christian Lubich, and Gerhard Wanner. *Geometric Numerical Integration: Structure Preserving Algorithms for Ordinary Differential Equations*. Springer, 2 edition, 2006.

[5] Joseph J. Monaghan. Smoothed particle hydrodynamics. *Annual Review of Astronomy and Astrophysics*, 30:543 to 574, 1992.

[6] Matthias Müller, David Charypar, and Markus Gross. Particle-based fluid simulation for interactive applications. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, page 154 to 159, 2003.

[7] NVIDIA Corporation. *CUDA C++ Programming Guide, version 13.3*, 2026.

[8] Lars Nyland, Mark Harris, and Jan Prins. Fast n-body simulation with CUDA. In Hubert Nguyen, editor, *GPU Gems 3*, chapter 31, page 677 to 695. Addison Wesley, 2007.

[9] Henry C. Plummer. On the problem of distribution in globular star clusters. *Monthly Notices of the Royal Astronomical Society*, 71:460 to 470, 1911.

[10] Matthias Teschner, Bruno Heidelberger, Matthias Müller, Danat Pomeranets, and Markus Gross. Optimized spatial hashing for collision detection of deformable objects. In *Proceedings of Vision, Modeling and Visualization*, page 47 to 54, 2003.

[11] Loup Verlet. Computer experiments on classical fluids: Thermodynamical properties of lennard-jones molecules. *Physical Review*, 159(1):98 to 103, 1967.